aerospace

MDPI

Article

# DRL-Based UAV Autonomous Navigation and Obstacle Avoidance with LiDAR and Depth Camera Fusion

**Bangsong Lei [1], Wei Hu [2,*], Zhaoxu Ren [1] and Shude Ji [1,*]**

[1] College of Aerospace Engineering, Shenyang Aerospace University, Shenyang 110136, China; raybs81@163.com (B.L.); renzx515@163.com (Z.R.)
[2] College of Automation, Shenyang Aerospace University, Shenyang 110136, China
* Correspondence: hwspeedcn@163.com (W.H.); superjsd@163.com (S.J.)

**Abstract**

With the growing application of unmanned aerial vehicles (UAVs) in complex, stochastic environments, autonomous navigation and obstacle avoidance represent critical technical challenges requiring urgent solutions. This study proposes an innovative deep reinforcement learning (DRL) framework that leverages multimodal perception through the fusion of LiDAR and depth camera data. A sophisticated multi-sensor data preprocessing mechanism is designed to extract multimodal features, significantly enhancing the UAV's situational awareness and adaptability in intricate, stochastic environments. In the high-level decision-maker of the framework, to overcome the intrinsic limitation of low sample efficiency in DRL algorithms, this study introduces an advanced decision-making algorithm, Soft Actor-Critic with Prioritization (SAC-P), which markedly accelerates model convergence and enhances training stability through optimized sample selection and utilization strategies. Validated within a high-fidelity Robot Operating System (ROS) and Gazebo simulation environment, the proposed framework achieved a task success rate of 81.23% in comparative evaluations, surpassing all baseline methods. Notably, in generalization tests conducted in previously unseen and highly complex environments, it maintained a success rate of 72.08%, confirming its robust and efficient navigation and obstacle avoidance capabilities in complex, densely cluttered environments with stochastic obstacle distributions.

**Keywords:** UAV; DRL; autonomous navigation; obstacle avoidance; LiDAR; depth camera; multi-sensor; ROS

check for updates

Academic Editor: Alexandre Brandão

Received: 23 July 2025
Revised: 16 September 2025
Accepted: 17 September 2025
Published: 20 September 2025

**Citation:** Lei, B.; Hu, W.; Ren, Z.; Ji, S. DRL-Based UAV Autonomous Navigation and Obstacle Avoidance with LiDAR and Depth Camera Fusion. *Aerospace* **2025**, *12*, 848. https://doi.org/10.3390/aerospace12090848

**Correction Statement:** This article has been republished with a minor change. The change does not affect the scientific content of the article and further details are available within the backmatter of the website version of this article.

**Copyright:** © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

## 1. Introduction

Advancements in edge computing and high-precision sensor technologies have significantly expanded the applications of unmanned aerial vehicles (UAVs) across domains such as surveillance [1], reconnaissance [2,3], search and rescue [4], logistics [5], environmental monitoring [6]. The ability to achieve robust autonomous navigation and effective obstacle avoidance is pivotal to ensuring safe, efficient, and reliable UAV operations in complex environments.

Conventional approaches to unmanned aerial vehicle (UAV) navigation and obstacle avoidance have demonstrated moderate efficacy in simple, static environments. For example, references [7,8] leveraged enhanced Rapidly exploring Random Tree (RRT) algorithms to facilitate UAV path planning, while references [9–11] utilized optimized A* algorithms for navigation and obstacle avoidance. Furthermore, bio-inspired heuristic

methods, including genetic algorithms [12,13], ant colony optimization (ACO) [14], and particle swarm optimization (PSO) [15,16], have been applied to address rudimentary UAV navigation and obstacle avoidance tasks. Nevertheless, these traditional methodologies exhibit notable limitations, including heavy reliance on prior environmental knowledge, elevated computational complexity, constrained perceptual capabilities, absence of adaptive learning mechanisms, and cumulative errors stemming from modular designs. Collectively, these deficiencies substantially undermine the robustness and generalizability of such approaches in complex, unknown environments.

To overcome the constraints of conventional approaches in UAV autonomous navigation and obstacle avoidance, learning-based algorithms have emerged as a promising solution for addressing challenges in complex environments. These algorithms leverage onboard sensors, such as monocular cameras, depth cameras, and LiDAR, to acquire environmental obstacle data [17], enabling autonomous obstacle avoidance through end-to-end decision-making frameworks. As outlined in [18], learning-based UAV navigation and obstacle avoidance algorithms are broadly classified into three categories: deep learning-based, reinforcement learning-based, and deep reinforcement learning (DRL)-based. Deep learning-based methods typically employ neural networks to process visual data from cameras, augmented by auxiliary sensors such as optical flow meters, to facilitate obstacle avoidance [19,20]. In contrast, reinforcement learning-based approaches rely on trial-and-error interactions with the environment, utilizing sensor inputs from LiDAR or cameras to inform decision-making networks, which generate optimal actions and refine strategies based on reward signals from environmental feedback [21,22]. By integrating the strengths of deep neural networks and reinforcement learning, DRL algorithms markedly enhance the UAV's capability for autonomous navigation and robust obstacle avoidance in intricate environments [23,24].

Despite advancements, DRL-based UAV navigation and obstacle avoidance research continues to encounter significant challenges. Firstly, the majority of studies depend on a single sensor modality, such as monocular cameras [25–27], depth cameras [28], or LiDAR [29,30], for environmental perception. In complex real-world settings, however, monocular and depth cameras are constrained by lighting conditions and limited fields of view, while LiDAR suffers from data sparsity, hindering comprehensive environmental awareness [31]. Secondly, DRL algorithms typically exhibit low sample efficiency, resulting in protracted policy convergence and limited adaptability to unknown environments.

In summary, current research on DRL-based UAV navigation exhibits several notable limitations: (1) a predominant reliance on a single sensing modality, which inherently restricts perception capability; (2) performance evaluations that are largely confined to sparse or simplified obstacle settings, rather than dense, unstructured, and stochastic environments that more faithfully reflect real-world conditions; and (3) insufficient emphasis on enhancing sample efficiency to enable faster and more adaptive policy learning. These constraints collectively impede the practical scalability and deployment of UAV navigation systems, where robust perception and efficient decision-making are paramount.

In addition, some existing integrated UAV system architectures have also explored similar directions. For example, ref. [32] combines intelligent onboard computing, multi-sensor cooperative perception, and efficient communication protocols to enhance UAV autonomy and mission performance. Therefore, to address the aforementioned limitations, this study proposes an innovative framework for UAV autonomous navigation and obstacle avoidance, leveraging the fusion of depth cameras and LiDAR to substantially enhance environmental perception in complex settings. An efficient data preprocessing mechanism is developed to extract robust features from multimodal sensor data, thereby strengthening the representation of environmental states. Furthermore, we introduce a

novel deep reinforcement learning algorithm, termed SAC-P, which integrates the Soft Actor-Critic (SAC) [33,34] framework with Prioritized Experience Replay (PER) [35]. By optimizing sample selection strategies, SAC-P markedly improves sample efficiency and elevates the overall performance of UAV navigation and obstacle avoidance. Simulation experiments validate that the proposed framework significantly enhances both sample efficiency and obstacle avoidance capabilities, with superior performance in densely cluttered environments featuring randomly distributed obstacles. The SAC-P-based high-level decision-making module demonstrates exceptional generalization and scalability across diverse scenarios, surpassing baseline algorithms. The primary contributions of this study are outlined as follows:

(1) A novel deep reinforcement learning (DRL)-driven framework for UAV autonomous navigation and obstacle avoidance is proposed. In this framework, a DRL algorithm is introduced into the high-level decision-making module as the core decision mechanism, while LiDAR and depth camera data are fused to achieve multimodal environmental perception. Considering the characteristics of different sensor data, a dedicated data preprocessing mechanism is designed to effectively extract multisource features, thereby significantly enhancing the UAV's perception robustness and obstacle avoidance performance in complex and stochastic environments.

(2) An efficient decision algorithm, SAC-P, is designed for the high-level decision-maker of the proposed framework. The algorithm combines the SAC method with the PER mechanism to optimize the sample selection strategy. The experimental results show that SAC-P not only significantly improves the sample utilization efficiency and model convergence speed, reduces the training time cost, but also shows better performance in UAV autonomous navigation and obstacle avoidance tasks.

(3) Comprehensive experimental validation was performed within a high-fidelity simulation environment leveraging ROS and Gazebo. The results confirm that the proposed multi-sensor fusion deep reinforcement learning framework delivers superior generalization and adaptability, efficiently executing autonomous navigation and obstacle avoidance tasks in complex, densely cluttered environments with randomly distributed obstacles, achieving performance that markedly outperforms baseline approaches.

The remainder of this paper is structured as follows: Section 2 outlines the foundational models of DRL and the methodology for constructing the UAV motion model. Section 3 provides a detailed exposition of the SAC-P algorithm's principles and implementation specifics, alongside a comprehensive description of the architecture for the DRL-based UAV autonomous navigation and obstacle avoidance framework, integrating LiDAR and depth camera perception. Section 4 presents the results of simulation experiments conducted with the proposed framework, accompanied by a thorough analysis and discussion. Section 5 concludes with a summary of the primary contributions and delineates avenues for future research.

## 2. Preliminaries

### 2.1. Partially Observable Markov Decision Process

In real-world unmanned aerial vehicle (UAV) autonomous navigation and obstacle avoidance tasks, the limited operational range of onboard depth camera and LiDAR sensors restricts the UAV to perceiving only partial environmental states. Consequently, as shown in Figure 1, the interaction between the UAV and the environment can be modeled as a partially observable Markov decision process (POMDP) [36]. POMDP is a mathematical

model used to describe the control decision process of an intelligent agent system in a partially observable environment, which can be represented as a tuple:

$$M_P = (S, A, P, R, O, \gamma). \tag{1}$$

where $S$ denotes the state space of the environment, with the environment state at each time step $t$ represented as $s_t$. Similarly, $A$ denotes the action space of the agent, with the action executed by the agent at each time step $t$ represented as $a_t$. Furthermore, $P : S \times A \times S \to [0, 1]$ is the state transition probability function, representing the probability of transitioning from state $s_t \in S$ to the next state $s_{t+1} \in S$ after executing action $a_t$, thus it can be expressed as $P(s_{t+1}|a_t, s_t)$. Additionally, $R : S \times A \to \mathbb{R}$ is the reward function, representing the immediate reward obtained by the agent for executing action $a_t$ in state $s_t \in S$. Moreover, $O$ indicates the observation space of the agent, with the partial observation received by the agent from the environment at each time step $t$ represented as $o_t$. Finally, $\gamma \in (0, 1)$ is the discount facto, which balances the significance of immediate versus future rewards.

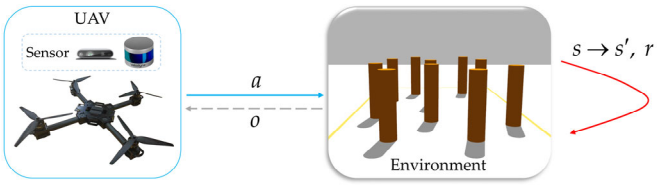


**Figure 1.** POMDP model.

### 2.2. Motion Model of UAV

This study employs a quadrotor UAV as the experimental platform. To facilitate subsequent modeling, this study establishes a world coordinate system $o - xyz$ and a UAV body coordinate system $o_b - x_b y_b z_b$, as depicted in Figure 2. The world coordinate system is defined using the East-North-Up (ENU) convention, while the body coordinate system adopts the Front-Left-Up (FLU) convention, with $x_b$, $y_b$ and $z_b$ corresponding to the UAV's forward, left, and upward directions, respectively.

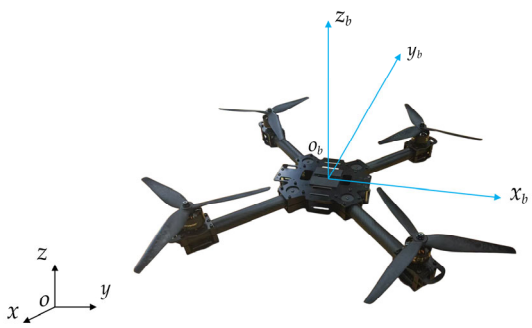


**Figure 2.** Coordinate system.

Typically, the control architecture for quadrotor UAVs comprises low-level and high-level control. The low-level control, managed by the flight controller, primarily governs the UAV's attitude stabilization. In contrast, the high-level control, executed by the onboard computer, focuses on integrating and processing data from various sensors and making advanced decisions using corresponding decision-making algorithms.

The motion of an UAV in three-dimensional space is characterized by six degrees of freedom. For simplicity, this study assumes that the UAV maintains a constant altitude during autonomous navigation and obstacle avoidance. Therefore, this study establishes

the UAV motion model in a two-dimensional Cartesian coordinate system, as shown in Figure 3, with the specific model described as follows:

$$\begin{cases} v_{t+1} = v_t + a_t \Delta_t \\ \phi_{t+1} = \phi + \omega_t \Delta_t \\ x_{t+1} = x_t + v_{t+1} \sin(\phi_{t+1}) \Delta_t \\ y_{t+1} = y_t + v_{t+1} \cos(\phi_{t+1}) \Delta_t \end{cases}. \tag{2}$$

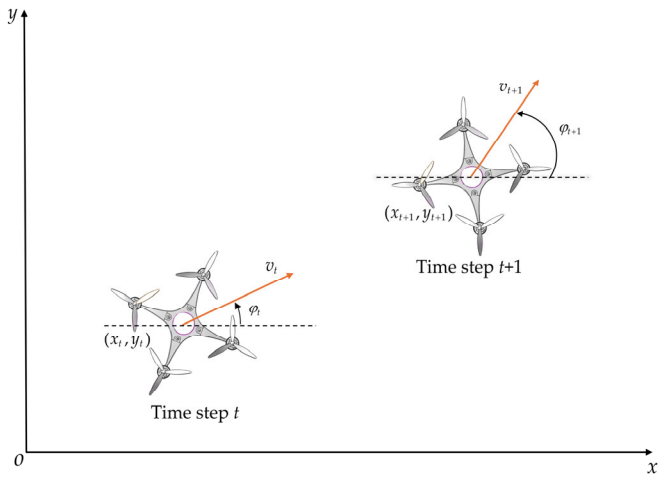


**Figure 3.** Motion model of UAV.

In the formulation, $v_t$ denotes the UAV's linear velocity along its heading at time step $t$, $\phi_t$ represents the yaw angle, defined as the angle between the body coordinate axis $x_a$ and the world coordinate axis $x$, with counterclockwise rotation about the body coordinate axis $z_b$ defined as positive, $\omega_t$ denotes the yaw angular velocity, $a_t$ represents the linear acceleration along the heading, $(x_t, y_t)$ indicates the global position of the UAV's center of mass in the world coordinate system, and $\Delta_t$ signifies the time interval between consecutive time steps.

## 3. Methodology

### *3.1. Problem Formulation*

In this section, the UAV autonomous navigation and obstacle avoidance task is formulated as a POMDP model. The UAV achieves end-to-end decision-making by integrating environmental data acquired from its onboard depth camera and LiDAR sensors to effectively avoid obstacles. Furthermore, it is assumed that the UAV can accurately acquire its pose information using onboard GPS and IMU sensors, and the target position information is known.

#### 3.1.1. Observation Space

In the actual task process, a UAV needs to decide which action to take to maximize the reward according to the current observation state. Therefore, this paper defines the observation space as follows:

$$O_t = \{o_{image,t}, o_{lidar,t}, o_{other,t}\}. \tag{3}$$

In the formulation, $o_{image,t} \in \mathbb{R}^{4\times64\times64}$ denotes the stacked depth image data, comprising four consecutive frames, acquired from the UAV's onboard binocular depth camera at time step $t$, Similarly, $o_{lidar,t} \in \mathbb{R}^{128}$ represents a 128-dimensional vector derived through down-sampling of the LiDAR sensor data. Additionally, $o_{other,t} \in \mathbb{R}^5$ is a vector

encapsulating relative information regarding the target position and the UAV's own state ($o_{other,t} = [d_t, \sin\theta_t^{diff}, \cos\theta_t^{diff}, v_t, \omega_t]$).

As illustrated in Figure 4, $d_t$ denotes the Euclidean distance between the UAV and the target position at time step $t$. The term $\theta_t^{diff}$ represents the angular deviation, defined as $\theta_t^{diff} = \phi_t - \phi_t$, where $\phi_t$ is the angle between the line connecting the UAV's center of mass to the target position and the world coordinate axis $x$, and $\phi_t$ is the UAV's current yaw angle. Additionally, $\sin\theta_t^{diff}$ and $\cos\theta_t^{diff}$ denote the sine and cosine of the deviation angle, respectively.

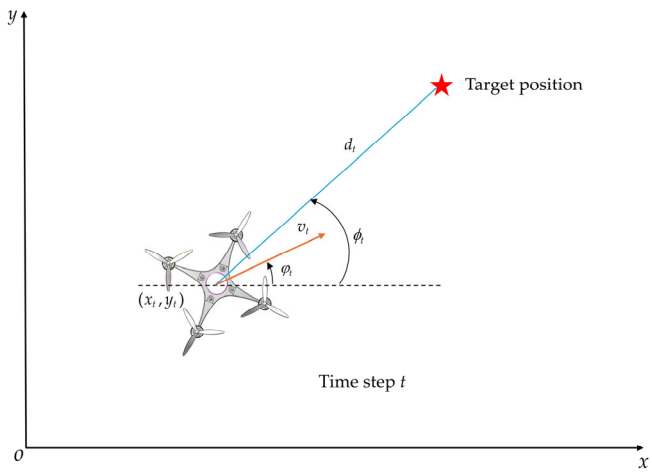


**Figure 4.** The relative position relationship between UAV and target position.

### 3.1.2. Action Space

This study focusses on the design of the high-level controller for the UAV, while the low-level control is implemented by the flight controller onboard the UAV. Typically, such flight controllers support various control modes, including position control, velocity control, and angular velocity control. Based on the UAV motion model presented in Equation (2), and accounting for the continuity of velocity variations, this study employs the linear velocity along the heading and the yaw angular velocity as control inputs to the low-level controller to achieve precise motion control of the UAV. Therefore, the action space of the UAV is defined as follows:

$$A_t = \{v_t, w_t\}. \tag{4}$$

### 3.1.3. Reword Function

In deep reinforcement learning, the design of the reward function plays a pivotal role in determining task performance. For the UAV autonomous navigation and obstacle avoidance task, this study designs a reward mechanism to encourage the UAV to move toward the target position while introducing a penalty function for erroneous action selections to optimize the control policy.

(1) The motion distance reward, defined as the reward based on the difference in distance ($\Delta d = d_{t-1} - d_t$) between the UAV and the target point from the previous time step to the current time step:

$$r_t^d = c_d \Delta d, \tag{5}$$

where $c_d$ denotes the weighting coefficient for the motion distance reward.

(2) The forward motion reward, designed to incentivize the UAV to progress along its heading while discouraging excessive backward motion:

$$r_t^p = \begin{cases} c_p(v_t/v_{\max}), if\ v_t \geq 0 \\ -c_p|v_t/v_{\max}|, if\ v_t < 0 \end{cases}, \tag{6}$$

where $c_p$ denotes the weighting coefficient for the forward motion reward.

(3) The heading alignment reward, designed to incentivize the UAV to adjust its heading to facilitate progress toward the target position:

$$r_t^{diff} = c_{diff} \cos \theta_t^{diff}, \tag{7}$$

where $c_{diff}$ is the heading deviation reward coefficient.

(4) The smoothness action reward, aimed at preventing abrupt changes in UAV actions, thereby ensuring the smoothness of the motion process:

$$r_t^s = c_s \Delta a, \tag{8}$$

where $c_s$ denotes the weighting coefficient for the action smoothness reward, and $\Delta a = ((v_t - v_{t-1})/v_{\max})^2 + ((w_t - w_{t-1})/w_{\max})^2$ represents the difference between the actions at the current and previous time steps.

(5) The task completion reward, awarded as a substantial positive reward when the UAV successfully avoids obstacles and reaches the target position:

$$r_t^m = c_m \cdot 200, \tag{9}$$

where $c_m$ denotes the coefficient of reward for task completion.

(6) The collision penalty, imposed as a substantial negative reward when the UAV collides with an obstacle:

$$r_t^c = \begin{cases} c_c \cdot (50/d_t^o), if\ d_{collision} < d_t^o < d_{precollision} \\ c_c \cdot 100, if\ d_t^o < d_{collision} \end{cases}, \tag{10}$$

where $d_t^o$ denotes the minimum obstacle distance detected by the LiDAR at time step $t$, while $d_{collision}$ and $d_{precollision}$ represent the predefined minimum collision distance and pre-collision distance, respectively, $c_c$ is the coefficient of collision penalty.

(7) The out-of-bounds penalty, imposed as a substantial negative reward when the UAV moves beyond the predefined task boundaries:

$$r_t^o = c_o \cdot 100, \tag{11}$$

where $c_o$ denotes the out-of-bound penalty coefficient.

(8) The task failure penalty, designed to deter the UAV from exhibiting suboptimal behaviors such as remaining stationary or moving within a limited range, imposes a substantial negative reward when the UAV reaches the maximum time step without colliding or exceeding boundaries:

$$r_t^f = c_f \cdot 100, \tag{12}$$

where $c_f$ denotes the penalty coefficient of task failure.

(9) The time penalty, designed to decrease as the UAV completes the task in less time, imposes a smaller negative reward for faster task completion:

$$r_t^{time} = c_{time} t, \tag{13}$$

where $c_{time}$ denotes the negative coefficient for the time penalty.

Based on the foregoing, the composite reward function for the UAV during the entire task is defined as follows:

$$R_t = r_t^d + r_t^p + r_t^{diff} + r_t^s + r_t^m + r_t^c + r_t^o + r_t^f + r_t^{time}. \tag{14}$$

### 3.2. UAV High-Level Decision-Making Algorithm

In this section, we present a detailed exposition of the high-level decision-making algorithm employed for autonomous UAV navigation and obstacle avoidance in this study. Given the continuous nature of the UAV's motion dynamics and the requirement for real-time processing of multi-sensor fusion data, we adopt the SAC algorithm for high-level decision-making. To enhance the convergence rate of the SAC algorithm, we incorporate the PER method.

#### 3.2.1. Soft Actor-Critic

The SAC algorithm is a state-of-the-art off-policy DRL method that leverages maximum entropy principles to achieve robust and sample-efficient policy optimization [33,34]. It is particularly well-suited for continuous control tasks. By balancing exploration and exploitation through entropy regularization, SAC ensures stable learning and adaptability in complex environments.

It operates within the maximum entropy reinforcement learning framework, augmenting the standard objective of maximizing expected cumulative rewards with an entropy regularization term. The objective is formalized as:

$$J(\pi) = \mathbb{E}_\pi \left[ \sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot|s_t))) \right], \tag{15}$$

where $\pi$ denotes the policy, $r(s_t, a_t)$ is the reward function, $\gamma \in (0, 1)$ is the discount factor, $\mathcal{H}(\pi(\cdot|s_t)) = -\mathbb{E}_{a_t \sim \pi}[\log \pi(a_t|s_t)]$ represents the entropy of the policy at state $s_t$, and $\alpha > 0$ is the temperature parameter balancing reward maximization and policy randomness.

The soft value functions are defined as follows. The state-action value function (Q-function) is:

$$Q^\pi(s_t, a_t) = r(s_t, a_t) + \gamma \mathbb{E}_{s_{t+1} \sim p}[V^\pi(s_{t+1})], \tag{16}$$

where the state value function $V^\pi(s_t)$ incorporates the entropy term:

$$V^\pi(s_t) = \mathbb{E}_{a_t \sim \pi}[Q^\pi(s_t, a_t) - \alpha \log \pi(a_t|s_t)]. \tag{17}$$

The optimal policy $\pi^*$ maximizes the expected Q-value minus the entropy penalty:

$$\pi^* = \underset{\pi}{\operatorname{argmax}} \mathbb{E}_{a \sim \pi}[Q^\pi(s, a) - \alpha \log \pi(a|s)]. \tag{18}$$

To parameterize the policy, SAC employs a Gaussian distribution, $\pi_\theta(a|s) = \mathcal{N}(\mu_\theta(s), \sigma_\theta(s))$, where $\theta$ denotes the policy network parameters. Actions are sampled using the repa-

rameterization trick, $a = f_\theta(s, \xi)$ with $\xi \sim \mathcal{N}(0, 1)$, enabling efficient gradient computation. The policy is optimized via the policy gradient:

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{s\sim\mathcal{D},\xi\sim\mathcal{N}}[\nabla_\theta \log \pi_\theta(a|s)(Q(s,a) - \alpha \log \pi_\theta(a|s))], \tag{19}$$

where $\mathcal{D}$ is the replay buffer storing past experiences.

For the critic, SAC employs two Q-networks, $Q_{\phi_1}$ and $Q_{\phi_2}$, to mitigate overestimation bias. The Q-function is optimized by minimizing the soft Bellman residual:

$$J_Q(\phi_i) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[\left(Q_{\phi_i}(s,a) - \left(r + \gamma \mathbb{E}_{a'\sim\pi}\left[\min_{j=1,2} Q_{\bar{\phi}_j}(s',a') - \alpha \log \pi(a' \mid s')\right]\right)\right)^2\right], \tag{20}$$

where $Q_{\bar{\phi}_j}$ are target Q-networks updated via exponential moving average (EMA) to stabilize training.

The temperature parameter $\alpha$ is adaptively tuned to ensure the policy's entropy approximates a target entropy $\mathcal{H}_0$. The objective for $\alpha$ is:

$$J(\alpha) = \mathbb{E}_{a\sim\pi}[-\alpha \log \pi(a|s) - \alpha\mathcal{H}_0]. \tag{21}$$

3.2.2. Prioritized Experience Replay

PER is an advanced sampling strategy for reinforcement learning that enhances sample efficiency by prioritizing experiences based on their learning utility [35]. Unlike uniform sampling in traditional experience replay, PER assigns higher sampling probabilities to transitions that are deemed more informative, typically measured by their temporal-difference (TD) error.

(1) Priority Assignment. In PER, the priority of a transition $i$, denoted $p_i$, is typically derived from its TD error, which quantifies the discrepancy between the predicted and target values in value-based reinforcement learning methods. For a transition $(s_i, a_i, r_i, s'_i)$, the TD error is defined as:

$$\delta_i = r_i + \gamma \max_{a'} Q(s'_i, a'_i) - Q(s_i, a_i), \tag{22}$$

where $Q(s_i, a_i)$ is the Q-value estimated by the current Q-network, and $Q(s'_i, a'_i)$ is the target Q-value computed using a target network. The priority is then defined as:

$$p_i = |\delta_i| + \epsilon, \tag{23}$$

where $\epsilon > 0$ is a small constant ensuring non-zero priorities for transitions with zero TD error, thus preventing sampling starvation.

(2) Sampling Probability. To prioritize informative transitions, PER employs a non-uniform sampling distribution where the probability of sampling transition $i$, denoted $P(i)$, is proportional to a transformed priority:

$$P(i) = \frac{p_i^\lambda}{\sum_j p_j^\lambda}, \tag{24}$$

where $\lambda \in [0, 1]$ is a hyperparameter controlling the degree of prioritization. When $\lambda = 0$, sampling reduces to uniform random sampling, while $\lambda = 1$ results in fully prioritized sampling proportional to the priorities. Typically, $\lambda \in (0, 1]$ is chosen to balance prioritization and exploration. Efficient sampling is achieved using a SumTree (Based on proportional priority) data structure, where leaf nodes store the priorities $p_i$,

and internal nodes maintain the cumulative sums of their children's priorities, enabling $O(logN)$ sampling complexity for a buffer of size $N$.

(3) Importance Sampling Correction. Non-uniform sampling introduces bias in the expected gradient updates, as the sampling distribution deviates from the underlying data distribution. To correct this, PER employs importance sampling weights, defined as:

$$w_i = \left(\frac{1/N}{P(i)}\right)^{\beta}, \tag{25}$$

where $N$ is the number of transitions in the replay buffer, and $\beta \in [0,1]$ is a hyperparameter controlling the degree of bias correction. When $\beta = 0$, no correction is applied, while $\beta = 1$ fully compensates for the sampling bias. In practice, $\beta$ is often annealed from a small value to 1 over training to ensure stability early on. The weights are applied to the loss function, typically the squared TD error:

$$L = w'_i \cdot \delta_i^2, \tag{26}$$

where $w'_i$ is normalized by the maximum weight in the mini-batch, $w'_i \leftarrow w_i / \max_j w_j$, to prevent gradient explosion and ensure numerical stability.

(4) Priority Updates. After each gradient update, the TD error of sampled transitions is recomputed to reflect changes in the Q-network. The updated TD error $\delta_i$ is used to revise the priority:

$$p_i \leftarrow |\delta_i| + \epsilon. \tag{27}$$

The SumTree is subsequently updated to maintain the correct priority distribution, ensuring that subsequent samplings reflect the latest learning dynamics.

### 3.2.3. SAC-P

This study designates the integrated algorithm as SAC-P. In contrast to the original SAC algorithm, the Q-network update incorporates importance sampling weights $w_i$, with the loss function defined as follows:

$$J_Q(\phi_i) = \mathbb{E}_{(s,a,r,s')\sim\mathcal{D}}\left[w'_i \cdot \left(Q_{\phi_i}(s,a) - \left(r + \gamma \mathbb{E}_{a'\sim\pi}\left[\min_{j=1,2} Q_{\overline{\phi}_j}(s',a') - \alpha \log \pi(a' \mid s')\right]\right)\right)^2\right]. \tag{28}$$

After each update, priorities are recomputed using the updated TD error and stored in the SumTree to reflect the current learning dynamics.

The complete SAC-P algorithm is described in Algorithm 1.

**Algorithm 1: SAC-P**

```
Initialize policy network π_θ
Initialize Q-networks Q_φ1, Q_φ2, and target Q-networks Q_φ̄1, Q_φ̄2
Initialize replay buffer D with a SumTree, PER hyperparameter λ and β
Initialize temperature α, soft update factor τ
Initialize the environment to obtain the initial state
for t = 1 to Total timesteps do
    execute action a, obtain s', r and termination
    store transition (s, a, r, s') in D with priority
    if t is greater than Time steps to start learning then
        extracting Minibatch samples from D
        update Q-networks by minimize Equation (28)
```

**Algorithm 1:** *Cont.*

```
        if t is divisible by Policy update frequency then
            update policy network by minimize Equation (19)
        end if
        update α minimize by Equation (21)
        if t is divisible by Target network update frequency then
            update target Q-networks by Q_{φ̄_j} ← τφ_j + (1 − τ)φ̄_j for (j = 1, 2)
        end if
        update priorities by Equation (27) and refresh the SumTree
        if termination is true then
            reset environment, s ← s′
        end if
    end for
```

### 3.3. System Framework

This subsection presents a detailed exposition of the framework for the UAV DRL lidar-vision navigation system developed in this study. It encompasses the convolutional neural network (CNN) encoder for preprocessing acquired depth images, the preprocessing of additional observations, and the operational principles of the overall system.

#### 3.3.1. CNN Encoder

Directly utilizing raw depth images from the depth camera as the observation state input for the policy network substantially increases its learning complexity. To mitigate this, we propose a CNN encoder to preprocess the depth image data, extracting essential features and incorporating them as part of the observation state for the policy network. This approach significantly reduces the learning complexity of the policy network. The architecture of the proposed CNN encoder, illustrated in Figure 5, comprises four convolutional layers, ReLU activation functions, average pooling layers, and residual connections. To address the limited field of view of the depth camera, we reshape a sequence of four consecutive depth image frames into 64 × 64 pixels images prior to encoding. The preprocessed depth images are sequentially processed through the network layers. The first and second layers employ 4 × 3 convolution kernels (stride 2, padding (2,1)), progressively reducing the spatial resolution from 64 × 64 to 16 × 16, and further to 8 × 8 via 2 × 2 average pooling, while increasing the feature channels from 4 to 64 to capture multi-scale spatial features. The third and fourth layers form a residual block with 3 × 3 convolution kernels (stride 1, padding 1), preserving the 8 × 8 × 64 spatial dimensions, leveraging residual connections to enhance feature representation and alleviate the vanishing gradient problem. Finally, the flattened feature vector is mapped to a 128-dimensional representation through a fully connected layer, integrated into the observation state for the policy network.

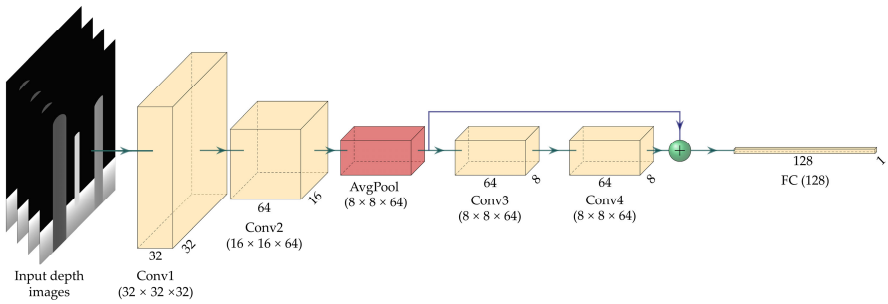


**Figure 5.** Network structure of the CNN encoder.

#### 3.3.2. Preprocessing of Additional Observation Data

In contrast to the intricate preprocessing required for depth image data, the preprocessing of other data in the observation space is comparatively straightforward and efficient. We propose a unified feature extraction framework wherein down-sampled lidar data is processed through a fully connected layer to extract essential features, mapping them to a 128-dimensional representation that forms part of the observation state for the policy network. Similarly, relative target position information and UAV state data are fused and processed through another fully connected layer, yielding a 128-dimensional feature representation, which is also incorporated into the observation state.

This approach ensures standardized processing and efficient fusion of multimodal observation data, enabling the policy network to effectively leverage features from diverse data sources, thereby enhancing learning performance.

#### 3.3.3. System Framework and Working Principle

Figure 6 illustrates the proposed system framework for autonomous UAV navigation and obstacle avoidance via DRL with fused LiDAR and depth camera information. Within this architecture, the UAV acquires real-time environmental state information through its integrated sensor suite: the onboard LiDAR and depth camera furnish spatial distribution information of obstacles, while the IMU and GPS sensors synchronously capture its own pose information. Based on these inputs, the relative target position information is calculated. These multidimensional sensory information streams are processed by dedicated preprocessing modules within the high-level decision-maker. Subsequently, the high-level decision-maker employs the preprocessed fused information to generate optimal action decisions using the SAC-P algorithm. These movement commands are then translated into actuation signals for individual execution units by the low-level controller, ultimately enabling autonomous obstacle evasion and precise waypoint navigation in unknown environments.

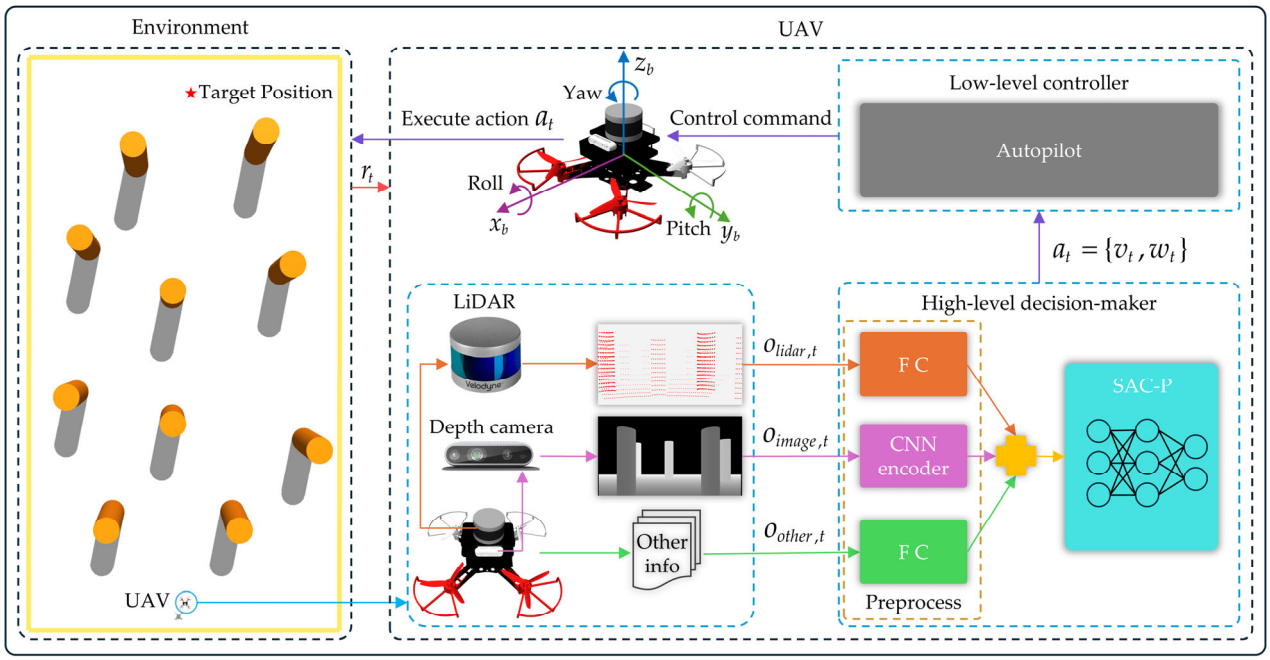


**Figure 6.** System framework.

## 4. Experiments and Analysis

### *4.1. Experiment Setting*

To facilitate joint simulation experiments for the UAV, this study leverages the Ubuntu 20.04 operating system, integrating the Robot Operating System (ROS) and Gazebo platforms for comprehensive simulation. ROS manages UAV flight control, while Gazebo provides a high-fidelity 3D simulation environment to replicate realistic physical interactions and stochastic scenes. This study adopts an open-source UAV simulator provided by Zhefan Xu [37] as the experimental platform. For sensor integration, we employ the D435 binocular depth camera and the Velodyne VLP-16 3D lidar, with detailed specifications outlined in Table 1. The hardware equipment specifications used in the experiments are detailed in Table 2.

**Table 1.** Sensor parameters.

| Sensor | Parameter Name | Value |
|---|---|---|
| D435i depth camera | Depth Detection Range | 0.3 m~10.0 m |
| | Depth Field of View | 87° × 58° |
| | Depth Resolution and FPS | 1280 × 720 30 fps |
| Velodyne VLP-16 LiDAR | Measuring Range | 0.9 m~130.0 m |
| | Vertical Field of View | −15°~15° |
| | Horizontal Field of View | 360° |

**Table 2.** Hardware equipment.

| Hardware Name | Value |
|---|---|
| CPU | Intel core i7–12700 KF |
| GPU | NVIDIA GeForce RTX 3060 (12 G) |
| RAM | 32 G |

#### 4.1.1. Scenario Setting

To comprehensively evaluate the performance of the proposed system framework in this study during task execution, a task area of 24 × 13 × 15 m was constructed in the Gazebo simulation environment, as shown in Figure 7. Obstacles are randomly distributed within the Obstacle subregion, with the UAV's starting and target positions randomly initialized on the left and right sides of this subregion, respectively. A task is deemed unsuccessful if the UAV collides with an obstacle, exits the task boundaries, or exceeds the maximum time step. The overall simulation effect is shown in Figure 8. This experimental design enables a systematic evaluation of the navigation system's robustness and effectiveness in partially observable environments.

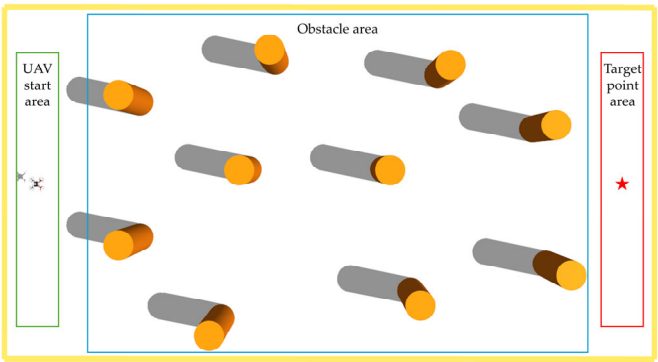


**Figure 7.** Simulation environment.

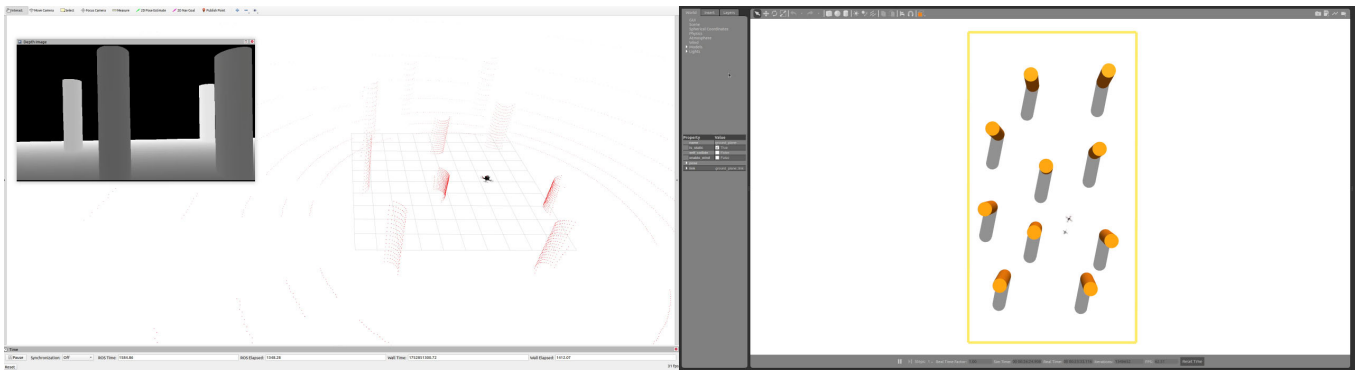

**Figure 8.** The overall simulation effect.

#### 4.1.2. Baseline and State Normalization

To assess the effectiveness of the proposed SAC-P algorithm, this study selected several state-of-the-art DRL algorithms as baselines for the high-level decision-making module, including Deep Deterministic Policy Gradient (DDPG) [38], Twin Delayed DDPG (TD3) [39], Proximal Policy Optimization (PPO) [40], and SAC. Notably, DDPG, TD3, and SAC are off-policy algorithms, whereas PPO is an on-policy algorithm. All the aforementioned baseline methods were evaluated within our proposed framework, employing an identical data preprocessing pipeline and the same sensor configuration. In addition, to compare performance with that of frameworks based on traditional algorithms, the APF [41] and DWA [42] algorithms were selected as baselines in this study.

To expedite policy learning and improve training stability, we normalize all observation states to ensure consistent numerical ranges. Specifically, depth image data $o_{\text{image},t}$, lidar data $o_{lidar,t}$, and the distance to the target $d_t$ are processed using a standardized normalization formula:

$$norm_data = \frac{data - \sigma_{\min}}{\sigma_{\max} - \sigma_{\min}} \tag{29}$$

where $\sigma_{\min}$ and $\sigma_{\max}$ are the predetermined minimum and maximum values for each data type, derived from hardware specifications or experimental analysis.

For the heading deviation states $\sin\theta_t^{diff}$ and $\cos\theta_t^{diff}$, which are naturally bounded within $[-1, 1]$, no further normalization is necessary. Moreover, the action space is defined with a maximum value of 1, such that the UAV's linear velocity $v_t$ and yaw rate $\omega_t$ require no additional normalization.

#### 4.1.3. Basic Parameter Settings

Following extensive testing and validation, the experimental parameter settings for this study are detailed in Table 3. Specifically, the maximum number of time steps per episode is set to $T_{\max} = 70$, the time interval between consecutive steps is $\Delta_t = 0.1$, and the reward function weights are configured as follows: movement distance $c_d = 10$, forward movement $c_p = 2$, heading deviation $c_{diff} = 5$, and action smoothness $c_s = -0.5$. Additionally, the minimum collision distance $d_{collision} = 1$ m, the target proximity threshold is $d_{pre} = 1.4$ m, and the time penalty weight is $c_{time} = -0.01$. The UAV's maximum linear velocity is $v_{\max} = 1$ m/s, and the maximum angular velocity is $w_{\max} = 1$ rad/s.

**Table 3.** Basic parameters.

| Parameter | Value | Parameter | Value |
|---|---|---|---|
| $T_{\max}$ | 70 | $c_o$ | $-1$ |
| $\Delta_t$ | 0.1 s | $c_f$ | $-1$ |

**Table 3.** *Cont.*

| Parameter | Value | Parameter | Value |
|---|---|---|---|
| $c_d$ | 10 | $c_{time}$ | −0.01 |
| $c_p$ | 2 | $d_{collision}$ | 1 m |
| $c_{diff}$ | 5 | $d_{pre}$ | 1.4 m |
| $c_s$ | −0.5 | $v_{\max}$ | 1 m/s |
| $c_m$ | 1 | $w_{\max}$ | 1 rad/s |
| $c_c$ | −1 | | |

### 4.2. Training

This study employs PyTorch 2.4.1 with CUDA 12.2 as the training framework. The hyperparameters for algorithm training are detailed in Table 4.

**Table 4.** Hyperparameters of Algorithm Training.

| Parameter | Value | Parameter | Value |
|---|---|---|---|
| *Total timesteps* | 1,000,000 | *Policy learning rate* | 0.0001 |
| *Buffer size* | 100,000 | *Critic learning rate* | 0.001 |
| *Minibatch* | 256 | $\tau$ | 0.005 |
| *Time steps tostartlearning* | 25,000 | $\gamma$ | 0.99 |
| **SAC-P** | | **TD3** | |
| *Policy update frequency* | 2 | *Policy noise* | 0.2 |
| *Target network update frequency* | 1 | *Exploration noise* | 0.1 |
| *Initial entropy coefficient* | 0.2 | *Policy update frequency* | 2 |
| $\lambda$ | 0.6 | *Noise clip* | 0.5 |
| $\beta$ | 0.4 | | |
| **SAC** | | **DDPG** | |
| *Policy noise* | 2 | *Exploration noise* | 0.1 |
| *Target network update frequency* | 1 | *Policy update frequency* | 2 |
| *Initial entropy coefficient* | 0.2 | *Noise clip* | 0.5 |
| **PPO** | | | |
| *Generaladvantageestimation*$\lambda^{ppo}$ | 0.95 | *Surrogate clipping coefficient* | 0.2 |
| *Minibatches* | 8 | *Entropy coefficient* | 0.01 |
| *Update epochs* | 10 | *Value function coefficient* | 0.5 |
| *Maximum gradient clippingnorm* | 0.5 | *Environment steps per rollout* | 2048 |
| *Target KL divergence threshold* | 0.015 | | |

In addition to the above hyperparameters, three different random seeds were used in this study to ensure the reliability of the results. At the initiation of each episode, obstacles were randomly distributed within predefined subregions to simulate diverse environmental conditions. The backbone architecture for the Actor and Critic networks across all evaluated algorithms consisted of a multilayer perceptron (MLP) with two hidden layers, each containing 256 units.

During the training phase, a fixed-size sliding window was utilized to compute the mean episodic reward, thereby enabling the analysis of reward trends over a specified temporal horizon. Figure 9 presents the average reward curves for all algorithms after training for $1 \times 10^6$ time steps, with shaded regions indicating the standard deviation derived from the three random seeds, thereby providing a measure of result variability.

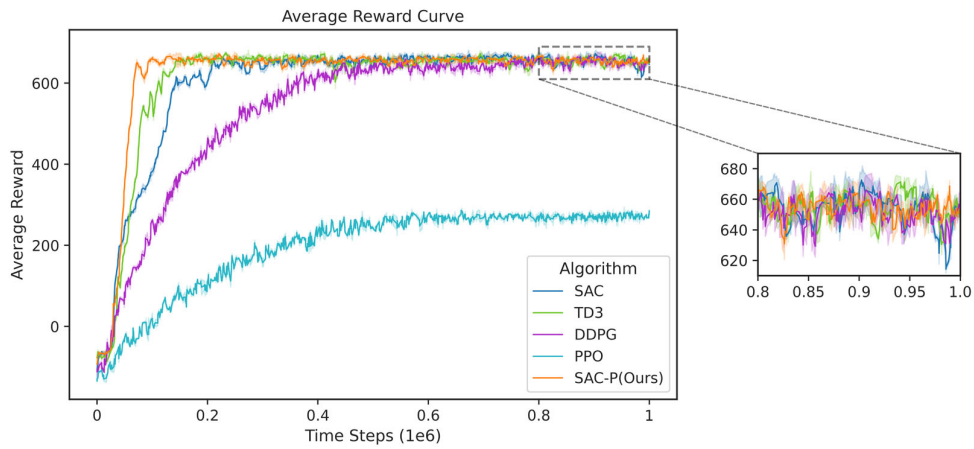


**Figure 9.** Average reward curve.

As illustrated in Figure 9, the proposed SAC-P algorithm achieves convergence at approximately $0.16 \times 10^6$ time steps, significantly outperforming the baseline algorithms. Specifically, TD3, SAC, DDPG, and PPO converge at approximately $0.20 \times 10^6$, $0.25 \times 10^6$, $0.50 \times 10^6$, and $0.60 \times 10^6$ time steps, respectively. These results underscore the superior convergence speed of the SAC-P algorithm. Furthermore, the experimental outcomes demonstrate that the optimized sampling strategy employed by SAC-P substantially enhances sample efficiency, yielding marked performance improvements over the baseline algorithms.

### *4.3. Testing*

#### 4.3.1. Performance Metrics

To evaluate the performance of the proposed SAC-P algorithm, we quantify its effectiveness using a comprehensive set of metrics: success rate, collision rate, out-of-bounds rate, timeout rate, step count, and flight distance.

(1) Success Rate (SR): The percentage of episodes in which the UAV successfully reaches the target position.

(2) Collision Rate (CR): The percentage of episodes where the UAV collides with an obstacle.

(3) Out-of-bound Rate (OR): The percentage of times the drone flies out of the mission area.

(4) Timeout Rate (TR): The percentage of episodes where the UAV fails to reach the target within the maximum time steps.

(5) Average Time Steps (ATS): The average number of steps required for the UAV to reach the target position from the starting position when the mission is successfully completed. In addition, since the energy consumed by the UAV during a mission is generally positively correlated with the number of time steps required, this study also considers the ATS as an indirect metric for evaluating the UAV's energy consumption.

#### 4.3.2. Comparative Testing

To enable an objective evaluation of the framework's performance across different algorithms, we designed the experiments using the controlled variable approach, thereby ensuring both fairness and consistency in testing conditions. In this setup, all algorithms are implemented within the same proposed framework, with the only variation being the decision-making algorithm; consequently, the data preprocessing procedure remains identical for all cases. Moreover, performance assessments are conducted under an identical obstacle distribution, where the obstacle coordinates are fixed at [[4.0, 4.0], [7.5, 2.0],

[−4.0, −5.0], [2.0, 0.5], [−7.0, 3.0], [3.0, −4.0], [−3.0, 0.5], [−2.0, 4.5], [−7.0, −2.0], [8.0, −3.0]]. The UAV's starting positions are set to three representative points: [−11.0, 0.0], [−11.0, 4.0], [−11.0, −4.0], with corresponding target positions at [11.0, 0.0], [11.0, −4.0], [11.0, 4.0]. Each start-target pair undergoes 100 independent trials, resulting in a total of 300 episodes. In addition, each experiment was independently repeated three times with different random seeds.

1. Comparison test with DRL baseline algorithms

Using the trained model, we first benchmarked our approach against representative DRL baselines in three distinct mission scenarios. Performance was evaluated using five metrics: SR, CR, OR, TR and ATS. Overall statistics are summarized in Table 5 (results are presented as mean ± standard deviation (SD)), with comparative SR performance illustrated in Figure 10. The low SD values—generally below ±2% for rate-based metrics and ±0.5 time steps for ATS—demonstrate that the performance differences are statistically stable and not caused by stochastic variations inherent to DRL training or UAV simulation environments.

**Table 5.** Performance comparison of cross-scene methods (with DRL baseline).

| Mission | Algorithm | SR (%) | CR (%) | OR (%) | TR (%) | ATS |
|---|---|---|---|---|---|---|
| 1 | DDPG | 69.23 ± 1.52 | 24.18 ± 1.21 | 2.05 ± 0.50 | 5.12 ± 0.58 | 62.85 ± 0.42 |
| | PPO | 72.11 ± 1.66 | 22.06 ± 1.28 | 2.04 ± 0.48 | 4.09 ± 0.46 | 61.42 ± 0.38 |
| | SAC | 74.02 ± 1.45 | 19.14 ± 1.10 | 1.08 ± 0.43 | 6.03 ± 0.52 | 62.25 ± 0.40 |
| | TD3 | 78.07 ± 1.78 | 12.09 ± 1.02 | 1.04 ± 0.39 | 9.08 ± 0.65 | 62.65 ± 0.41 |
| | SAC-P (Ours) | **81.23 ± 1.20** | **15.02 ± 1.01** | **1.02 ± 0.40** | **3.01 ± 0.49** | **61.26 ± 0.35** |
| 2 | DDPG | 61.16 ± 1.57 | 34.12 ± 1.48 | 2.06 ± 0.47 | 3.02 ± 0.38 | 68.32 ± 0.50 |
| | PPO | 60.08 ± 1.54 | 37.18 ± 1.59 | 1.02 ± 0.41 | 2.03 ± 0.33 | 68.51 ± 0.52 |
| | SAC | 66.04 ± 1.68 | 21.16 ± 1.33 | 5.08 ± 0.59 | 8.01 ± 0.71 | 67.01 ± 0.48 |
| | TD3 | 67.07 ± 1.44 | 26.08 ± 1.25 | 5.02 ± 0.58 | 2.04 ± 0.40 | 66.89 ± 0.46 |
| | SAC-P (Ours) | **77.15 ± 1.28** | **18.04 ± 1.02** | 2.01 ± 0.48 | 3.05 ± 0.41 | **66.33 ± 0.42** |
| 3 | DDPG | 61.05 ± 1.49 | 32.10 ± 1.37 | 3.01 ± 0.50 | 4.07 ± 0.46 | 68.83 ± 0.47 |
| | PPO | 57.03 ± 1.61 | 38.07 ± 1.54 | 1.05 ± 0.39 | 4.06 ± 0.49 | 68.61 ± 0.49 |
| | SAC | 66.08 ± 1.42 | 24.12 ± 1.21 | 2.07 ± 0.49 | 8.02 ± 0.63 | 67.67 ± 0.45 |
| | TD3 | 70.11 ± 1.51 | 20.05 ± 1.11 | 5.01 ± 0.57 | 5.03 ± 0.58 | 67.21 ± 0.44 |
| | SAC-P (Ours) | **76.13 ± 1.26** | **17.07 ± 1.03** | 2.02 ± 0.47 | 5.04 ± 0.48 | **66.22 ± 0.40** |

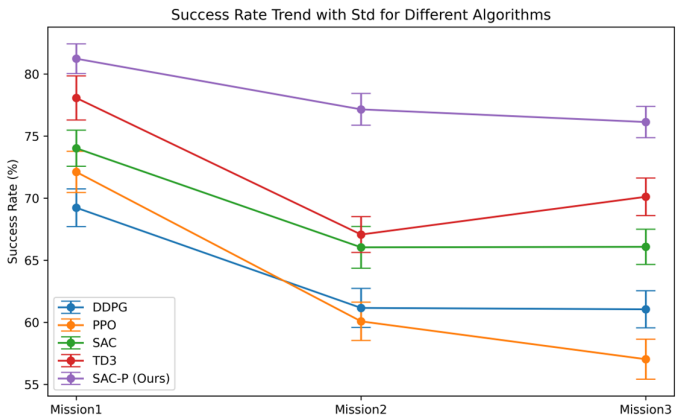


**Figure 10.** The success rate of different algorithms for each mission (with DRL baseline).

In Mission 1, the SAC-P-based framework achieved the highest performance across all metrics (SR = 81.23 ± 1.20%, CR = 15.02 ± 1.01%, OR = 1.02 ± 0.40%, TR = 3.01 ± 0.49%, ATS = 61.26 ± 0.35), outperforming the strongest baseline TD3 (SR = 78.07 ± 1.78%) by

approximately 3.16 percentage points. Notably, SAC-P required the fewest time steps, completing missions ~1.59 steps faster than DDPG, while maintaining low collision and failure rates. The small SD values (<±1.3%) indicate that these improvements were consistently observed across all trial runs.

Mission 2 results further validated the robustness of SAC-P. Its maintained superiority with an SR of 77.15 ± 1.28%, exceeding TD3 by ~10 percentage points. CR dropped to 18.04 ± 1.02%, a relative reduction of 16.08 and 19.14 percentage points compared with DDPG (34.12 ± 1.48%) and PPO (37.18 ± 1.59%), respectively. Although OR (2.01 ± 0.48%) and TR (3.05 ± 0.41%) were slightly higher than PPO's values, they remained negligible in operational terms. SAC-P achieved the shortest ATS (66.33 ± 0.42), highlighting its navigation efficiency and path-planning robustness. Low SD values again support the reproducibility of these gains.

In Mission 3, SAC-P achieved SR = 76.13 ± 1.26%, maintaining a 6.02 percentage point margin over TD3. It recorded the lowest CR (17.07 ± 1.03%), dramatically outperforming PPO (38.07 ± 1.54%) and DDPG (32.10 ± 1.37%). Although OR (2.02 ± 0.47%) and TR (5.04 ± 0.48%) were marginally higher than PPO in certain scenarios, they remained within acceptable limits for safe UAV navigation. SAC-P again required the fewest average steps (66.22 ± 0.40). The consistently small SD values confirm that performance differences were not due to random instability.

Taken together, results from all three missions demonstrate that the SAC-P framework consistently attains the dual advantage of the highest SR and shortest ATS. The statistical stability, as evidenced by small standard deviations across three independent runs, underscores the robustness and reproducibility of SAC-P in UAV autonomous navigation and obstacle avoidance tasks. Algorithmically, SAC-P preserves exploratory behavior while achieving more stable learning of environmental dynamics, enabling a favorable balance between success rate and collision avoidance. The comparison of the UAV trajectories of the proposed method with the baselines of different DRL algorithms in each task is shown in Figures 11a, 11b and 11c, respectively.

2. Comparative test with the baseline of Non-DRL algorithm

To further assess how the proposed SAC-P-based framework and decision-making strategy perform relative to conventional navigation and obstacle avoidance approaches in mission execution, we compared it against counterparts implemented with the APF and DWA algorithms under the same experimental settings described earlier. The overall results are reported in Table 6, with success rate comparisons illustrated in Figure 12. A detailed analysis of the outcomes for the three mission scenarios is presented as follows.

**Table 6.** Performance comparison of cross-scene methods (with Non-DRL baseline).

| Mission | Algorithm | SR (%) | CR (%) | OR (%) | TR (%) | ATS |
|---|---|---|---|---|---|---|
| 1 | SAC-P (Ours) | **81.23 ± 1.20** | **15.02 ± 1.01** | **1.02 ± 0.40** | **3.01 ± 0.49** | **61.26 ± 0.35** |
| | APF | 72.15 ± 1.65 | 22.08 ± 1.28 | 2.03 ± 0.46 | 4.06 ± 0.51 | 65.35 ± 0.44 |
| | DWA | 70.14 ± 1.58 | 23.06 ± 1.25 | 2.05 ± 0.49 | 5.03 ± 0.52 | 66.76 ± 0.47 |
| 2 | SAC-P (Ours) | **77.15 ± 1.28** | **18.04 ± 1.02** | **2.01 ± 0.48** | **3.05 ± 0.41** | **66.33 ± 0.42** |
| | APF | 66.10 ± 1.50 | 26.05 ± 1.27 | 3.04 ± 0.50 | 5.01 ± 0.53 | 68.14 ± 0.45 |
| | DWA | 67.12 ± 1.46 | 25.07 ± 1.29 | 4.06 ± 0.54 | 4.05 ± 0.51 | 68.05 ± 0.46 |
| 3 | SAC-P (Ours) | **76.13 ± 1.26** | **17.07 ± 1.03** | **2.02 ± 0.47** | **5.04 ± 0.48** | **66.22 ± 0.40** |
| | APF | 68.08 ± 1.49 | 24.04 ± 1.26 | 2.01 ± 0.46 | 6.02 ± 0.55 | 68.91 ± 0.44 |
| | DWA | 66.05 ± 1.44 | 24.06 ± 1.27 | 3.07 ± 0.51 | 7.03 ± 0.57 | 69.56 ± 0.45 |

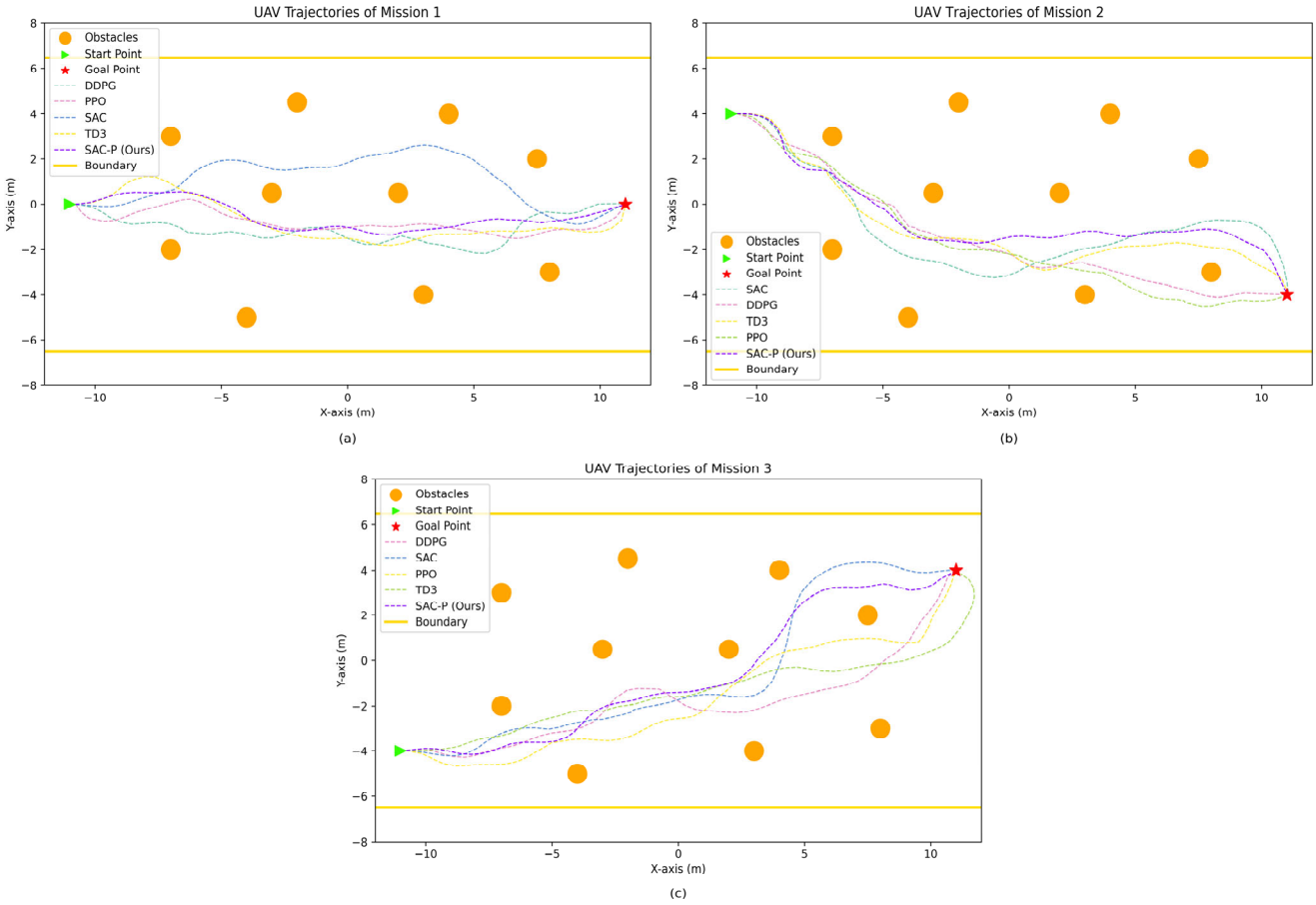


**Figure 11.** The UAV's motion trajectories in each task (with DRL baseline): (**a**) mission 1; (**b**) mission 2; (**c**) mission 3.

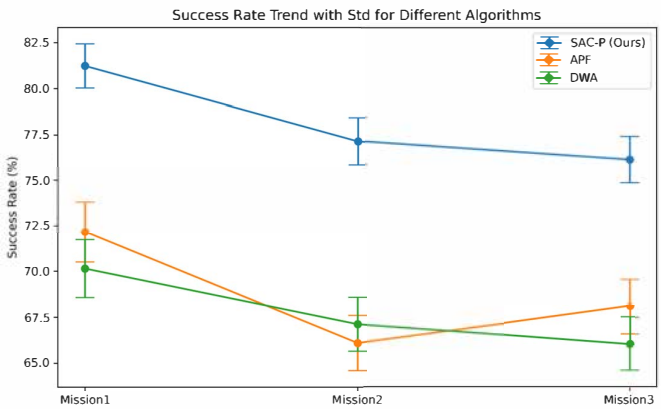


**Figure 12.** The success rate of different algorithms for each mission (with Non-DRL baseline).

In Mission 1, the SAC-P achieved the highest SR (81.23 ± 1.20%), representing a relative improvement of 9.08 and 11.09 percentage points over APF (72.15 ± 1.65%) and DWA (70.14 ± 1.58%), respectively. The CR was limited to 15.02 ± 1.01%, significantly lower than APF (22.08 ± 1.28%) and DWA (23.06 ± 1.25%). Both OR (1.02 ± 0.40%) and TR (3.01 ± 0.49%) were lower than those of the baselines. SAC-P also achieved the shortest ATS (61.26 ± 0.35), outperforming APF by 4.09 steps and DWA by 5.50 steps. The consistently low SD values confirm that these performance gains were stable across repeated runs, underscoring SAC-P's robustness and superior navigation efficiency in this mission.

In the more challenging Mission 2, SAC-P maintained its dominance with an SR of 77.15 ± 1.28%, outperforming APF (66.10 ± 1.50%) by 11.05 percentage points and DWA

(67.12 ± 1.46%) by 10.03 percentage points. It also recorded the lowest CR (18.04 ± 1.02%), OR (2.01 ± 0.48%), and TR (3.05 ± 0.41%) among all methods, indicating reduced mission failure risks. Furthermore, SAC-P achieved the shortest ATS (66.33 ± 0.42), which translates into 1.72 and 1.61 fewer steps than APF and DWA, respectively. The narrow SD margins again confirm that its performance advantages were consistent and not the result of experimental variance.

For Mission 3, SAC-P sustained its lead with an SR of 76.13 ± 1.26%, compared to APF (68.08 ± 1.49%) and DWA (66.05 ± 1.44%). Its CR (17.07 ± 1.03%) was substantially lower than that of APF (24.04 ± 1.26%) and DWA (24.06 ± 1.27%). Although APF achieved a marginally lower OR (2.01 ± 0.46% vs. SAC-P's 2.02 ± 0.47%), SAC-P retained the advantage in TR (5.04 ± 0.48%) and ATS (66.22 ± 0.40), outperforming APF and DWA by 2.69 and 3.34 steps, respectively. This reflects SAC-P's ability to retain high success rates while minimizing travel time even in stochastic scenarios. Across all missions, the SAC-P-based UAV navigation and obstacle avoidance framework consistently achieved the highest SR and delivered optimal results across CR, OR, TR, and ATS. Compared with APF- and DWA-based counterparts, it demonstrated clear advantages in mission efficiency, safety, and robustness. These outcomes affirm the method's superior performance in completing tasks efficiently, reducing failure likelihood, and maintaining robust operation under identical environmental conditions. The comparison of the UAV trajectories of the proposed method with the baselines of different Non-DRL algorithms in each task is shown in Figure 13a, Figure 13b, Figure 13c, respectively.

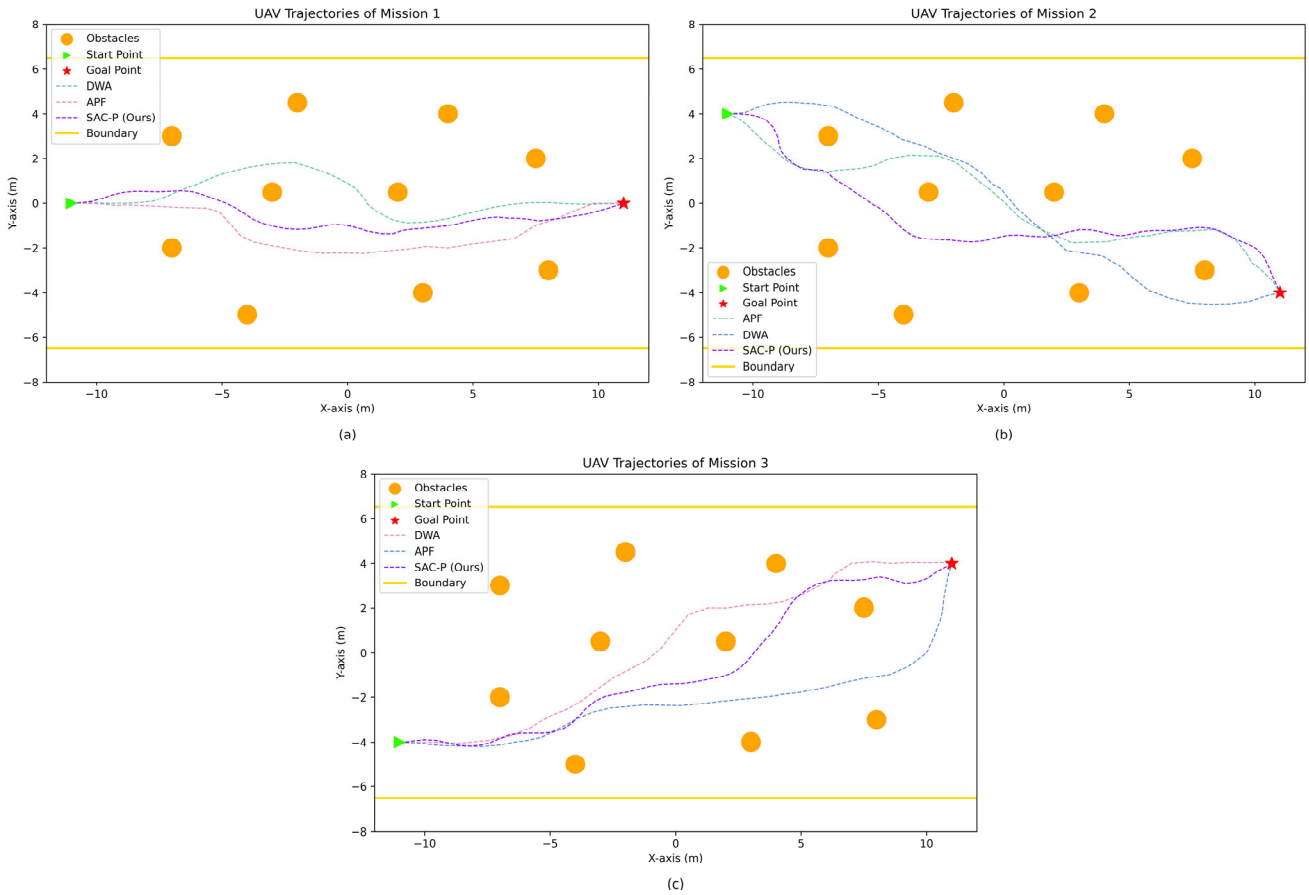


**Figure 13.** The UAV's motion trajectories in each task (with Non-DRL baseline): (**a**) mission 1; (**b**) mission 2; (**c**) mission 3.

#### 4.3.3. Generalization Performance Testing

To examine the generalization capability of the proposed SAC-P-based UAV navigation and obstacle avoidance framework, we performed a dedicated generalization test. A single experiment comprised 100 episodes, which were repeated three times independently with different random seeds. At the start of each episode, the obstacle configuration was randomly regenerated. To analyze performance across varying conditions, we designed two distinct task scenarios. Scenario 1 featured sparsely distributed homogenous obstacles. Scenario 2, inspired by the configurations in [39,40], contained a dense arrangement of heterogeneous obstacles with circular, square, and rectangular cross-sections. Because the initial and target positions, as well as obstacle arrangements, varied between episodes, the required number of time steps also varied. As such, the evaluation focused exclusively on four metrics: SR, CR, OR, and TR.

3. Simple Obstacle Scenario Test

Table 7 summarizes the results for Scenario 1, where the proposed SAC-P-based autonomous navigation and obstacle avoidance framework was evaluated in an environment featuring randomly distributed, homogeneous obstacles. achieved an SR of 79.12 $\pm$ 1.85%, with CR (17.06 $\pm$ 1.42%), OR (2.01 $\pm$ 0.45%), and TR (2.04 $\pm$ 0.47%) all maintained at low levels. The relatively small SD values ($\leq\pm$1.85%) indicate that performance remained stable and reproducible across all unseen map configurations, demonstrating strong adaptability to novel environments. Importantly, the task settings, obstacle positions, and spatial configurations in this scenario were entirely unseen during training, making the results a direct measure of the framework's adaptability and transferability to unfamiliar environments.

**Table 7.** Generalization performance test results (Scenario 1).

| Algorithm | SR (%) | CR (%) | OR (%) | TR (%) |
|---|---|---|---|---|
| SAC-P | 79.12 $\pm$ 1.85 | 17.06 $\pm$ 1.42 | 2.01 $\pm$ 0.45 | 2.04 $\pm$ 0.47 |

The combination of low CR and OR indicates that the UAV could reliably detect and avoid obstacles under a completely new map topology, while the low TR reflects efficient path planning and decision-making even in untrained scenarios. Together, these findings underscore the framework's strong generalization capability, demonstrating that navigation policies learned in training can be effectively and robustly applied to new, more uncertain task conditions. Representative UAV trajectories and corresponding obstacle configurations for selected episodes in Scenario 1 are depicted in Figure 14.

4. Complex Obstacle Scenario Test

As environmental complexity increases—particularly when obstacles are heterogeneous, densely packed, and randomly distributed—UAV navigation and obstacle avoidance place heightened demands on environmental perception, path planning, and decision stability. Table 8 reports the results for Scenario 2.

**Table 8.** Generalization performance test results (Scenario 2).

| Algorithm | SR (%) | CR (%) | OR (%) | TR (%) |
|---|---|---|---|---|
| SAC-P | 72.08 $\pm$ 2.14 | 12.04 $\pm$ 1.63 | 2.02 $\pm$ 0.48 | 4.01 $\pm$ 0.52 |

The SAC-P-based framework achieved an SR of 72.08 $\pm$ 2.14%, lower than the 79.12% attained in Scenario 1, reflecting the increased difficulty of trajectory planning in the presence of dense and irregular obstacle fields. The CR rose to 22.04 $\pm$ 1.68% (+5 percentage points compared with Scenario 1), indicating that denser obstacle configurations increased the frequency

of local avoidance maneuvers and collision risk. Nevertheless, both OR (2.02 ± 0.48%) and TR (4.01 ± 0.52%) remained low, demonstrating that the framework could maintain operational safety and execution stability even in high-uncertainty environments.

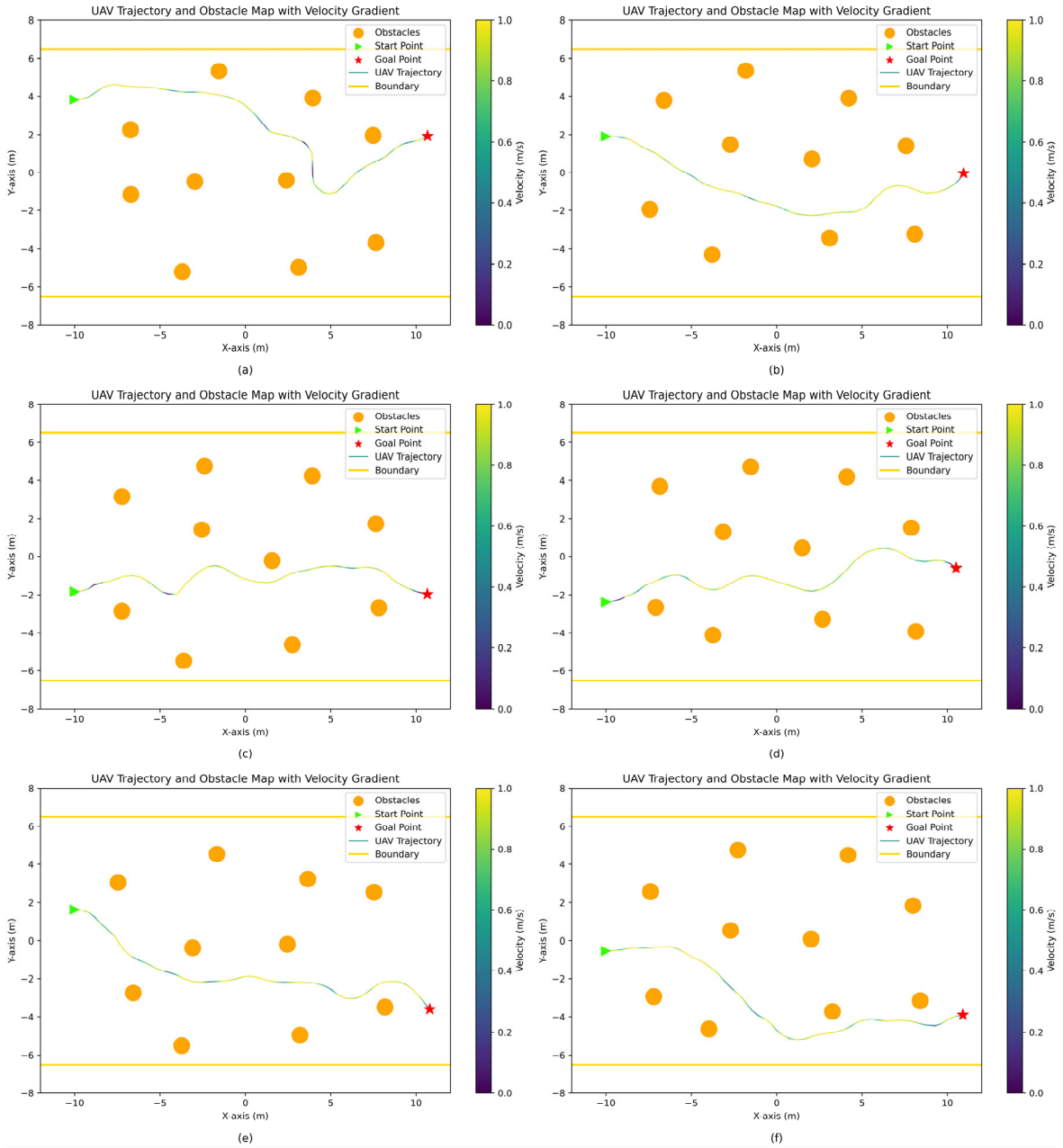


**Figure 14.** The UAV trajectories and their corresponding obstacle distributions (Scenario 1): (**a**–**f**) denote different test episodes, respectively.

The relatively small SD values across all metrics (≤±2.14%) confirm that these results are statistically stable, and that SAC-P's performance degradation from Scenario 1 to Scenario 2 is moderate and consistent, rather than the result of isolated failures. This indicates that the proposed framework preserves robust generalization capability and reliable decision-making in complex, previously unseen operational contexts.

Trajectory analysis in Figure 15 further reveals that, even in the complex scenario, the UAV maintained smooth path generation without excessive turning or futile backtracking. This indicates that the framework's high-level decision module can preserve stable, efficient global planning despite pronounced local environmental changes, avoiding both

decision oscillation and overly conservative maneuvers. Collectively, these results reinforce the framework's robustness in maintaining path continuity and decision consistency in challenging environments, confirming its strong global planning capabilities.

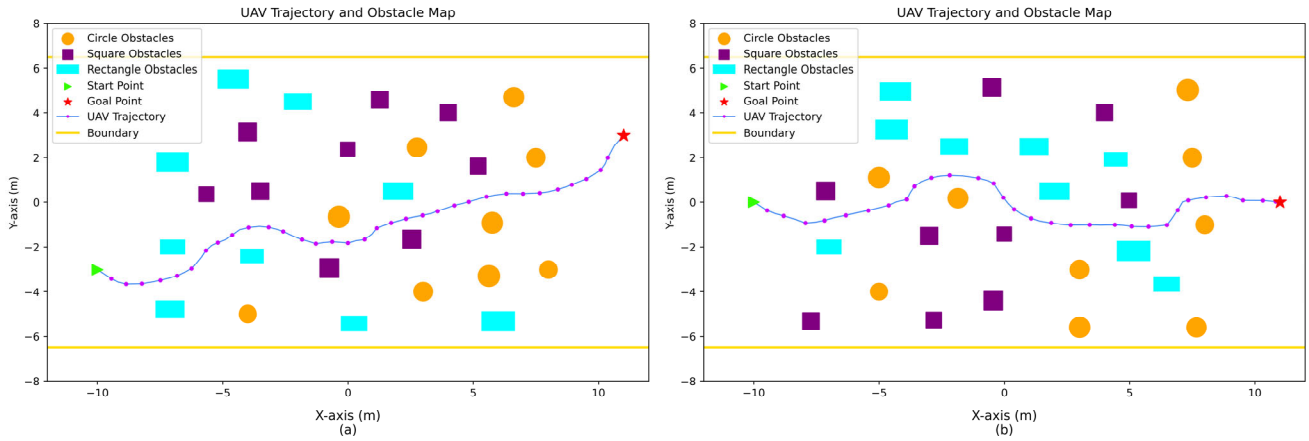


**Figure 15.** The UAV trajectories and their corresponding obstacle distributions (Scenario 2): (**a**,**b**) denote different test episodes, respectively.

#### 4.3.4. Performance Evaluation of Single-Sensor and Sensor Fusion Frameworks

To assess the relative merits of the proposed UAV DRL framework—which fuses depth camera and LiDAR perception—against single-sensor counterparts, we performed a sensor configuration comparison while keeping the decision-making algorithm (SAC-P) fixed across all tests. Three input configurations for environmental observation were evaluated: depth camera only, LiDAR only, and the fusion of depth camera with LiDAR. The same preprocessing pipeline was applied to all sensory data to ensure fairness and comparability, and the experimental scenario was configured identically to Mission 1 in the aforementioned comparative tests, with each sensor configuration tested over 100 episodes. Each experiment was independently repeated three times with different random seeds. Performance was evaluated across six metrics: SR, CR, OR, TR, ATS, and an additional measure—Average single-step decision time (ASDT)—to gauge real-time decision-making efficiency.

The results, summarized in Table 9, show that sensor selection substantially affects mission performance even under identical decision-making policies. The consistently small SD values (generally $\leq\pm1.8\%$ for rate-based metrics, $\leq\pm0.5$ steps for ATS, and $\leq\pm0.004$ s for ASDT) indicate stable and reproducible performance across runs. The proposed Depth camera + LiDAR fusion configuration achieved the highest success rate (SR) of $81.23 \pm 1.20\%$, outperforming the Depth camera–only setup ($68.15 \pm 1.65\%$) by 13.08 percentage points and the LiDAR–only setup ($72.14 \pm 1.58\%$) by 9.09 percentage points. It also yielded the lowest collision rate (CR) ($15.02 \pm 1.01\%$), minimal out-of-bounds rate (OR) ($1.02 \pm 0.40\%$), and the lowest timeout rate (TR) ($3.01 \pm 0.49\%$) among all configurations.

**Table 9.** Performance comparison of frames with different sensors.

| Sensor | SR (%) | CR (%) | OR (%) | TR (%) | ATS | ASDT (s) |
|---|---|---|---|---|---|---|
| Depth camera + LiDAR (ours) | **81.23 ± 1.20** | **15.02 ± 1.01** | **1.02 ± 0.40** | **3.01 ± 0.49** | **61.26 ± 0.35** | 0.043 ± 0.002 |
| Depth camera | 68.15 ± 1.65 | 26.08 ± 1.28 | 2.03 ± 0.46 | 4.06 ± 0.51 | 64.54 ± 0.44 | 0.031 ± 0.002 |
| LiDAR | 72.14 ± 1.58 | 22.06 ± 1.25 | 3.05 ± 0.49 | 3.04 ± 0.52 | 63.15 ± 0.40 | 0.029 ± 0.002 |

In terms of navigation efficiency, the fusion system achieved the shortest average time steps (ATS) (61.26 ± 0.35), reducing task completion time by 2.89 steps compared with Depth camera–only and by 1.89 steps compared with LiDAR–only configurations. This improvement reflects the enhanced environmental awareness and more optimal route selection achievable through the complementary sensing modalities. Regarding decision-making latency, the fusion setup exhibited a slightly higher ASDT (0.043 ± 0.002 s) than the single-sensor configurations (Depth camera: 0.031 ± 0.002 s, LiDAR: 0.029 ± 0.002 s), attributable to the additional computational overhead from sensor fusion and feature integration. However, this latency increase is negligible relative to the UAV control cycle and does not adversely affect real-time execution.

Overall, these findings confirm that combining depth camera and LiDAR perception within the proposed DRL framework delivers superior performance and robustness over single-sensor approaches, enabling more reliable, higher-success navigation in complex and uncertain environments.

## 5. Conclusions

### 5.1. Overall Conclusions

This work addresses the challenge of autonomous navigation and obstacle avoidance for UAVs operating in complex, stochastic environments by proposing a deep reinforcement learning (DRL) framework that fuses multimodal perception from LiDAR and depth cameras. The framework is designed to deliver both high efficiency and robustness in navigation tasks. To enhance perception accuracy and reliability, we developed a multi-sensor data preprocessing pipeline that extracts multimodal features, significantly improving the UAV's ability to perceive and adapt to intricate obstacle distributions. To overcome the inherent inefficiency of DRL in sample utilization, we introduce an improved SAC-P algorithm as the framework's high-level decision-making module, which accelerates convergence and enhances training stability through more effective sample usage.

High-fidelity simulation experiments built on the ROS and Gazebo platforms were conducted to comprehensively validate the performance of the proposed framework. Experimental results demonstrate that the SAC-P-based framework significantly outperforms existing baseline algorithms in key performance metrics, such as mission success rate, collision rate, and average time steps, showcasing its efficient navigation and obstacle avoidance performance in complex stochastic obstacle environments. These findings provide important theoretical and technical support for UAV autonomous operation in unknown environments.

### 5.2. Future Work

In future work, we plan to incorporate more real-world environmental factors into the modeling and algorithm design of the autonomous navigation and obstacle avoidance framework, including wind field disturbances, dynamic obstacles, GPS signal drift, delay of UAV controller and multisource sensor noise. For such challenging conditions, further improvements will be made to the high-level decision-making algorithm, along with the design of effective compensation and robustness enhancement mechanisms, to strengthen the system's adaptability and stability in dynamic and unstructured environments. Furthermore, we will extend the framework from static to dynamic scenarios and systematically verify its generalization capability and anti-interference performance under real mission conditions through Sim-to-Real transfer experiments. In addition, efforts will be devoted to deeply optimizing computational efficiency to meet the strict latency requirements of real-time navigation, thereby providing more reliable technical support for UAV deployment in complex real-world environments.

In addition, we also plan to extend the proposed framework from single-UAV navigation to multi-UAV cooperative missions, leveraging robust depth camera–LiDAR fusion perception, an efficient decision-making mechanism, and reliable inter-UAV communication to achieve swarm-level cooperative navigation and obstacle avoidance in complex environments.

**Author Contributions:** Conceptualization, B.L. and Z.R.; methodology, B.L.; software, B.L.; validation, B.L., W.H. and S.J.; formal analysis, B.L. and Z.R.; investigation, B.L.; resources, S.J.; data curation, W.H.; writing—original draft preparation, B.L.; writing—review and editing, S.J. and W.H.; visualization, B.L.; supervision, S.J.; project administration, W.H.; funding acquisition, Z.R. All authors have read and agreed to the published version of the manuscript.

**Funding:** This research was funded by Qingdao Key Technology Research and Industrialization Demonstration Project of Qingdao City, Shandong Province, China, grant number 24-1-3-hygg-12-hy.

**Data Availability Statement:** Data is contained within the article.

**Conflicts of Interest:** The authors declare no conflicts of interest.

## References

1. Alam, M.M.; Moh, S. Joint Topology Control and Routing in a UAV Swarm for Crowd Surveillance. *J. Netw. Comput. Appl.* **2022**, *204*, 103427. [CrossRef]
2. Liu, W.; Zhang, T.; Huang, S.; Li, K. A Hybrid Optimization Framework for UAV Reconnaissance Mission Planning. *Comput. Ind. Eng.* **2022**, *173*, 108653. [CrossRef]
3. Xie, C.; Wu, B.; Guo, D.; Chen, X. Autonomous Navigation UAVs for Enhancing Information Freshness for Reconnaissance. *Electronics* **2024**, *13*, 1354. [CrossRef]
4. Martinez-Alpiste, I.; Golcarenarenji, G.; Wang, Q.; Alcaraz-Calero, J.M. Search and Rescue Operation Using UAVs: A Case Study. *Expert. Syst. Appl.* **2021**, *178*, 114937. [CrossRef]
5. Lee, S.Y.; Han, S.R.; Song, B.D. Simultaneous Cooperation of Refrigerated Ground Vehicle (RGV) and Unmanned Aerial Vehicle (UAV) for Rapid Delivery with Perishable Food. *Appl. Math. Modell.* **2022**, *106*, 844–866. [CrossRef]
6. Vazquez-Carmona, E.; Vasquez-Gomez, J.; Herrera-Lozada, J. Environmental Monitoring Using Embedded Systems on UAVs. *IEEE Lat. Am. Trans.* **2020**, *18*, 303–310. [CrossRef]
7. Li, M.; Sun, Q.; Zhu, M. UAV 3-Dimension Flight Path Planning Based on Improved Rapidly-Exploring Random Tree. In Proceedings of the 2019 Chinese Control and Decision Conference (CCDC), Nanchang, China, 3–5 June 2019; pp. 921–925.
8. Yang, H.; Jia, Q.; Zhang, W. An Environmental Potential Field Based RRT Algorithm for UAV Path Planning. In Proceedings of the 2018 37th Chinese Control Conference (CCC), Wuhan, China, 25–27 July 2018; pp. 9922–9927.
9. Primatesta, S.; Guglieri, G.; Rizzo, A. A Risk-Aware Path Planning Strategy for UAVs in Urban Environments. *J. Intell. Rob. Syst.* **2019**, *95*, 629–643. [CrossRef]
10. Wu, X.; Xu, L.; Zhen, R.; Wu, X. Bi-Directional Adaptive A* Algorithm toward Optimal Path Planning for Large-Scale UAV under Multi-Constraints. *IEEE Access* **2020**, *8*, 85431–85440. [CrossRef]
11. Zhang, Z.; Wu, J.; Dai, J.; He, C. A Novel Real-Time Penetration Path Planning Algorithm for Stealth UAV in 3D Complex Dynamic Environment. *IEEE Access* **2020**, *8*, 122757–122771. [CrossRef]
12. Pehlivanoglu, Y.V. A New Vibrational Genetic Algorithm Enhanced with a Voronoi Diagram for Path Planning of Autonomous UAV. *Aerosp. Sci. Technol.* **2012**, *16*, 47–55. [CrossRef]
13. Roberge, V.; Tarbouchi, M.; Labonté, G. Fast Genetic Algorithm Path Planner for Fixed-Wing Military UAV Using GPU. *IEEE Trans. Aerosp. Electron. Syst.* **2018**, *54*, 2105–2117. [CrossRef]
14. Chen, J.; Ye, F.; Jiang, T. Path Planning under Obstacle-Avoidance Constraints Based on Ant Colony Optimization Algorithm. In Proceedings of the 2017 IEEE 17th International Conference on Communication Technology (ICCT), Chengdu, China, 27–30 October 2017; pp. 1434–1438.
15. Liu, Y.; Zhang, X.; Guan, X.; Delahaye, D. Adaptive Sensitivity Decision Based Path Planning Algorithm for Unmanned Aerial Vehicle with Improved Particle Swarm Optimization. *Aerosp. Sci. Technol.* **2016**, *58*, 92–102. [CrossRef]
16. Wang, H.; Lyu, W.; Yao, P.; Liang, X.; Liu, C. Three-Dimensional Path Planning for Unmanned Aerial Vehicle Based on Interfered Fluid Dynamical System. *Chin. J. Aeronaut.* **2015**, *28*, 229–239. [CrossRef]
17. Butt, M.Z.; Nasir, N.A.; Rashid, R.B. A Review of Perception Sensors, Techniques, and Hardware Architectures for Autonomous Low-Altitude UAVs in Non-Cooperative Local Obstacle Avoidance. *Rob. Auton. Syst.* **2024**, *173*, 104629.

18. Sharma, G.; Jain, S.; Sharma, R.S. Path Planning for Fully Autonomous UAVs-a Taxonomic Review and Future Perspectives. *IEEE Access* **2025**, *13*, 13356–13379. [CrossRef]
19. Back, S.; Cho, G.; Oh, J.; Tran, X.-T.; Oh, H. Autonomous UAV Trail Navigation with Obstacle Avoidance Using Deep Neural Networks. *J. Intell. Rob. Syst.* **2020**, *100*, 1195–1211. [CrossRef]
20. Müller, H.; Niculescu, V.; Polonelli, T.; Magno, M.; Benini, L. Robust and Efficient Depth-Based Obstacle Avoidance for Autonomous Miniaturized UAVs. *IEEE Trans. Rob.* **2023**, *39*, 4935–4951. [CrossRef]
21. Aburaya, A.; Selamat, H.; Muslim, M.T. Review of Vision-Based Reinforcement Learning for Drone Navigation. *Int. J. Intell. Rob. Appl.* **2024**, *8*, 974–992.
22. AlMahamid, F.; Grolinger, K. Autonomous Unmanned Aerial Vehicle Navigation Using Reinforcement Learning: A Systematic Review. *Eng. Appl. Artif. Intell.* **2022**, *115*, 105321. [CrossRef]
23. Yang, S.; Yu, G.; Meng, Z.; Wang, Z.; Li, H. Autonomous Obstacle Avoidance of UAV Based on Deep Reinforcement Learning. *J. Intell. Fuzzy Syst.* **2022**, *42*, 3323–3335. [CrossRef]
24. Wang, F.; Zhu, X.; Zhou, Z.; Tang, Y. Deep-Reinforcement-Learning-Based UAV Autonomous Navigation and Collision Avoidance in Unknown Environments. *Chin. J. Aeronaut.* **2024**, *37*, 237–257. [CrossRef]
25. Zhang, Y.; Yan, C.; Xiao, J.; Feroskhan, M. NPE-DRL: Enhancing Perception Constrained Obstacle Avoidance with Nonexpert Policy Guided Reinforcement Learning. *IEEE Trans. Artif. Intell.* **2025**, *6*, 184–198. [CrossRef]
26. Zhang, Y.; Low, K.H.; Lyu, C. Partially-Observable Monocular Autonomous Navigation for UAV through Deep Reinforcement Learning. In Proceedings of the AIAA Aviation 2023 Forum; American Institute of Aeronautics and Astronautics, San Diego, CA, USA, 12–16 June 2023.
27. Lin, H.-Y.; Peng, X.-Z. Autonomous Quadrotor Navigation with Vision Based Obstacle Avoidance and Path Planning. *IEEE Access* **2021**, *9*, 102450–102459. [CrossRef]
28. Wang, D.; Li, W.; Liu, X.; Li, N.; Zhang, C. UAV Environmental Perception and Autonomous Obstacle Avoidance: A Deep Learning and Depth Camera Combined Solution. *Comput. Electron. Agric.* **2020**, *175*, 105523. [CrossRef]
29. Rubí, B.; Morcego, B.; Pérez, R. Quadrotor Path Following and Reactive Obstacle Avoidance with Deep Reinforcement Learning. *J. Intell. Rob. Syst.* **2021**, *103*, 62. [CrossRef]
30. Yang, Y.; Hou, Z.; Chen, H.; Lu, P. DRL-Based Path Planner and Its Application in Real Quadrotor with LIDAR. *J. Intell. Rob. Syst.* **2023**, *107*, 38. [CrossRef]
31. Kolar, P.; Benavidez, P.; Jamshidi, M. Survey of Datafusion Techniques for Laser and Vision Based Sensor Integration for Autonomous Navigation. *Sensors* **2020**, *20*, 2180. [CrossRef]
32. Bakirci, M. A Novel Swarm Unmanned Aerial Vehicle System: Incorporating Autonomous Flight, Real-Time Object Detection, and Coordinated Intelligence for Enhanced Performance. *Trait. Signal* **2023**, *40*, 2063–2078. [CrossRef]
33. Haarnoja, T.; Zhou, A.; Abbeel, P.; Levine, S. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. *arXiv* **2018**, arXiv:1801.01290.
34. Haarnoja, T.; Zhou, A.; Hartikainen, K.; Tucker, G.; Ha, S.; Tan, J.; Kumar, V.; Zhu, H.; Gupta, A.; Abbeel, P.; et al. Soft Actor-Critic Algorithms and Applications 2019. *arXiv* **2018**, arXiv:1812.05905.
35. Schaul, T.; Quan, J.; Antonoglou, I.; Silver, D. Prioritized Experience Replay 2016. *arXiv* **2015**, arXiv:1511.05952.
36. Oliehoek, F.A.; Amato, C. *A Concise Introduction to Decentralized POMDPs*; SpringerBriefs in Intelligent Systems; Springer International Publishing: Cham, Switzerland, 2016; pp. 29–30. ISBN 978-3-319-28927-4.
37. Xu, Z. Uav Simulator. Available online: https://github.com/Zhefan-Xu/NavRL/tree/main/ros1/uav_simulator (accessed on 21 July 2025).
38. Lillicrap, T.P.; Hunt, J.J.; Pritzel, A.; Heess, N.; Erez, T.; Tassa, Y.; Silver, D.; Wierstra, D. Continuous Control with Deep Reinforcement Learning. *arXiv* **2015**, arXiv:1509.02971.
39. Fujimoto, S.; van Hoof, H.; Meger, D. Addressing Function Approximation Error in Actor-Critic Methods. *arXiv* **2018**, arXiv:1802.09477. [CrossRef]
40. Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; Klimov, O. Proximal Policy Optimization Algorithms. *arXiv* **2017**, arXiv:1707.06347. [CrossRef]
41. Khatib, O. Real-Time Obstacle Avoidance for Manipulators and Mobile Robots. In Proceedings of the 1985 IEEE International Conference on Robotics and Automation Proceedings, St. Louis, MO, USA, 25–28 March 1985; Volume 2, pp. 500–505.
42. Fox, D.; Burgard, W.; Thrun, S. The Dynamic Window Approach to Collision Avoidance. *IEEE Robot. Autom. Mag.* **1997**, *4*, 23–33. [CrossRef]


**Disclaimer/Publisher's Note:** The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.